Construcción de Software

Tema 10: Diseño de la arquitectura

Ing. Antonio Arroyo-Paz

Escuela Profesional de Ingeniería de Sistemas · UNSA · Semestre 2026-B

Información de la sesión

SEMANA

10

26/10/2026 – 31/10/2026

Tema 10: Diseño de la arquitectura

Curso	Construcción de Software
Docente	Ing. Antonio Arroyo-Paz
Semestre / Grupo	2026-B · Grupos de Mar y Mié
Guía de laboratorio	N.º 08 — Diseño de la arquitectura
Modalidad	Teoría 100 min · Inicio, Utilidad, Transformación, Práctica, Cierre

Agenda

1. Inicio · 10 min
2. Utilidad · 10 min
3. Transformación · 50 min
4. Práctica grupal · 25 min
5. Cierre · 5 min

Logro de aprendizaje

Al finalizar la sesión, el estudiante **diseña** la arquitectura de un sistema a partir de escenarios de atributos de calidad, con patrones arquitectónicos y tácticas, y la evalúa con un árbol de utilidad.

1 Comprende

Explica el proceso ADD, los escenarios de calidad, los patrones arquitectónicos y las tácticas.

2 Aplica

Realiza en grupo una iteración de ADD para RutaColca y la registra en un ADR.

3 Evalúa

Identifica riesgos y trade-offs de un diseño con un ATAM ligero.

Resultado del estudiante (ABET): C.c — Diseña sistemas con restricciones realistas; C.d — Trabaja en equipo; C.g — Comunica a públicos diversos.

01

SECCIÓN

Inicio

10 min · Saberes previos: ¿aguantará AgroConecta la campaña navideña?

¿Aguantará la campaña navideña?

- **Recordemos:** ADR-001 monolito modular (Tema 06), C4 (Tema 08) y patrones (Tema 09).
- **Situación:** en diciembre suben los pedidos, la pasarela falla a ratos y SUNAT se demora.
- **Reto en parejas (3 min):** ¿qué decisión de arquitectura tomarían en cada caso?



Figura 1. Tres situaciones de la campaña navideña de AgroConecta (ejemplo).

02

SECCIÓN

Utilidad

10 min · Qué es diseñar la arquitectura, cuáles son sus entradas y qué dice la investigación.

¿Qué es diseñar la arquitectura?

Diseño de la arquitectura

Proceso de tomar decisiones que transforman los drivers —propósito, funciones principales, atributos de calidad y restricciones— en estructuras de software que los satisfacen.

— Adaptado de Cervantes y Kazman (2016) y Bass et al. (2021).

Drivers

Decisiones

Estructuras

Las entradas del diseño: los drivers

- **Propósito:** MVP en 1 mes para la cooperativa.
- **Funciones principales:** publicar, pedir y pagar.
- **Atributos de calidad:** escenarios medibles.
- **Restricciones:** Java 21, PostgreSQL, equipo de 4.
- **Preocupaciones:** registro, pruebas y despliegue.

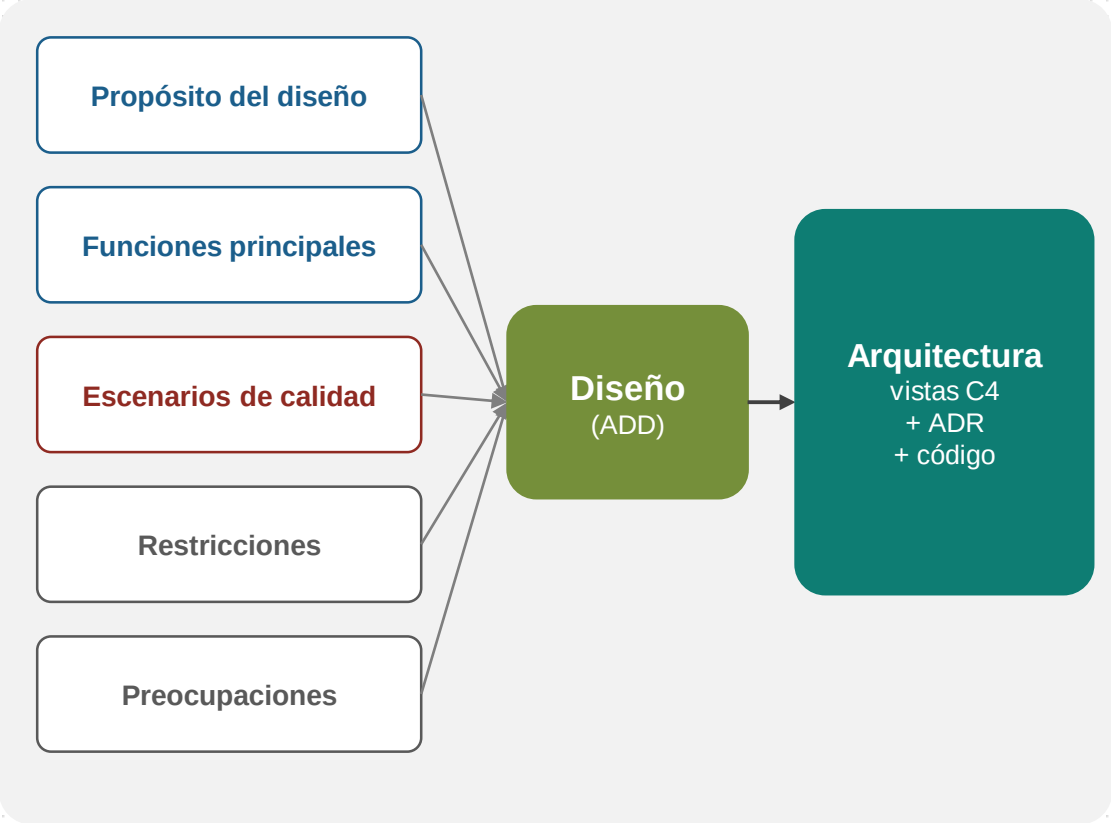


Figura 2. Entradas y salidas del diseño de arquitectura. Fuente: adaptado de Cervantes y Kazman (2016).

Estado del Arte: diseño de arquitectura e IA

Autor (año)	Propuesta	Aporte principal	Relación con el tema
Jansen y Bosch (2005)	La arquitectura como conjunto de decisiones de diseño	Si no se registran, las decisiones y su razonamiento se pierden	Registrar cada decisión en un ADR
Cervantes et al. (2013)	Uso de frameworks dentro del diseño con ADD	Los frameworks se eligen como conceptos de diseño, igual que los patrones	Spring Boot también es una decisión de diseño
Dhar et al. (2024)	Generación de decisiones de arquitectura con LLM	GPT-4 propone decisiones relevantes, aún por debajo del nivel humano	La IA ayuda a redactar el ADR; el equipo decide
Schmid et al. (2025)	Revisión sistemática de LLM en arquitectura (18 estudios)	Clasifican decisiones, detectan patrones y proponen diseños	Faltan estudios sobre conformidad con la arquitectura
Esposito et al. (2026)	Revisión multivocal de IA generativa en arquitectura	Apoyo a decisiones; riesgos de alucinación y evaluación débil	Toda propuesta de la IA se evalúa con escenarios

Búsqueda en IEEE Xplore, ScienceDirect, ACM y arXiv (2005–2026).

03

SECCIÓN

Transformación

50 min · Tema 1: escenarios y ADD. Tema 2: patrones arquitectónicos. Tema 3: tácticas. Tema 4: evaluación.

Escenario de atributo de calidad

- **Para qué:** convertir "debe ser rápido" en algo medible.
- **Seis partes:** fuente, estímulo, artefacto, entorno, respuesta y medida.
- **Clave:** sin medida no se puede verificar.
- **Origen:** Bass et al. (2021).

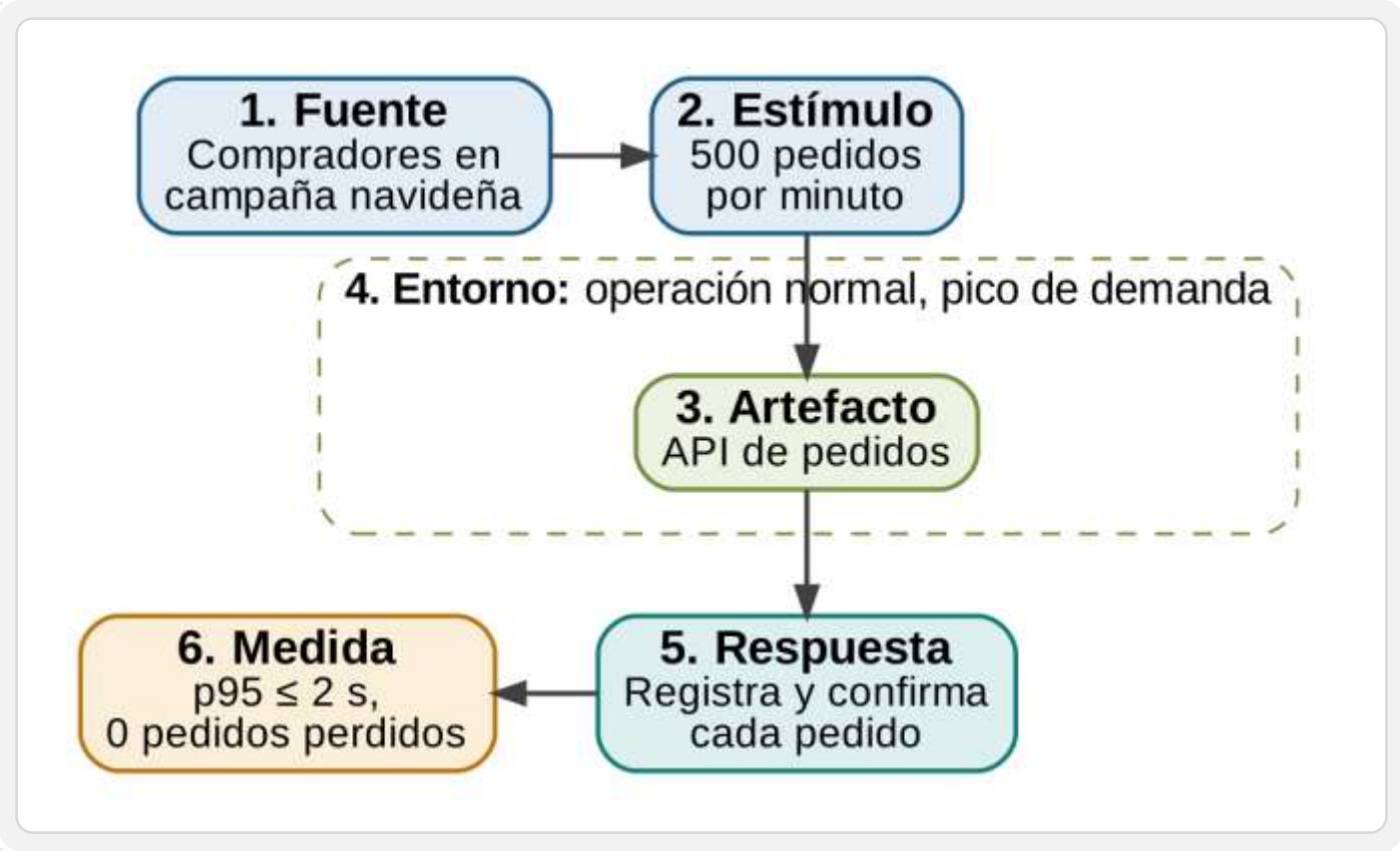


Figura 3. Partes de un escenario de rendimiento de AgroConecta (ejemplo).

Escenarios de calidad (ejemplo)

Atributo	Escenario (estímulo → respuesta)	Medida de respuesta
Rendimiento	500 pedidos por minuto en la campaña → la API registra y confirma cada pedido	p95 ≤ 2 s; 0 pedidos perdidos
Disponibilidad	La pasarela no responde 10 min → los pedidos quedan PENDIENTE_PAGO y se reintentan	0 pagos perdidos; cobro ≤ 15 min tras la recuperación
Modificabilidad	La cooperativa pide un nuevo medio de pago → se agrega un adaptador	1 clase nueva, ≤ 2 días, 0 cambios en el dominio
Seguridad	Un usuario intenta ver pedidos de otro productor → acceso denegado	100 % de intentos bloqueados (403) y auditados
Interoperabilidad	SUNAT (SEE) no responde al emitir un comprobante → se encola y reintenta	Comprobante emitido ≤ 24 h

Attribute-Driven Design (ADD 3.0)

- **Qué es:** un método iterativo para diseñar a partir de los drivers.
- **Cada iteración:** pocos drivers, conceptos de diseño y decisiones.
- **Conceptos:** patrones, tácticas y frameworks.
- **Salida:** vistas C4 y ADR.

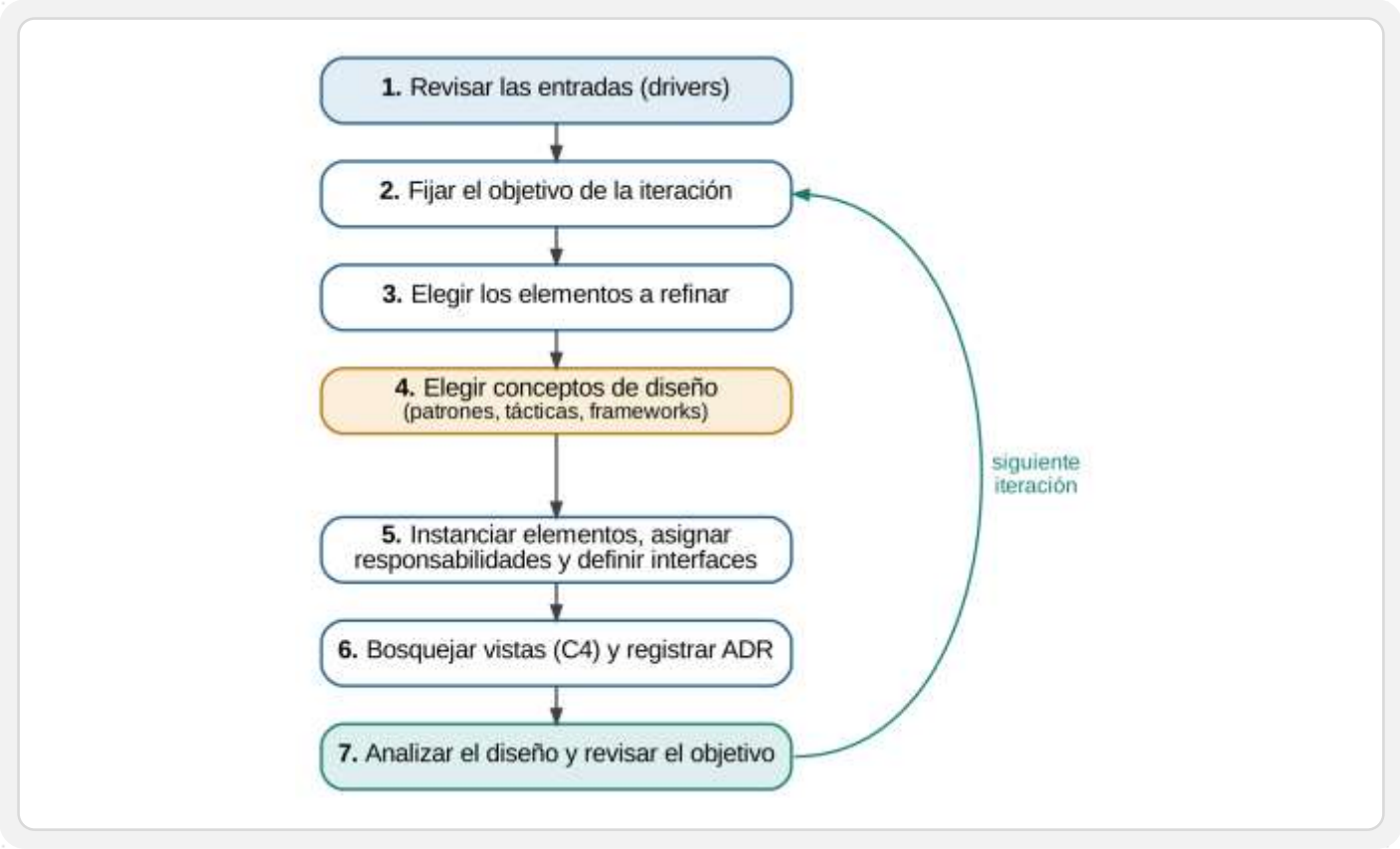


Figura 4. Los siete pasos de ADD 3.0. Fuente: adaptado de Cervantes y Kazman (2016).

Arquitectura hexagonal

- **Idea:** el núcleo (dominio y casos de uso) no depende de frameworks ni de la BD.
- **Puertos:** interfaces que define el dominio.
- **Adaptadores:** REST, JPA, Yape (Adaptador del Tema 09).
- **Favorece:** pruebas y cambio de tecnología.

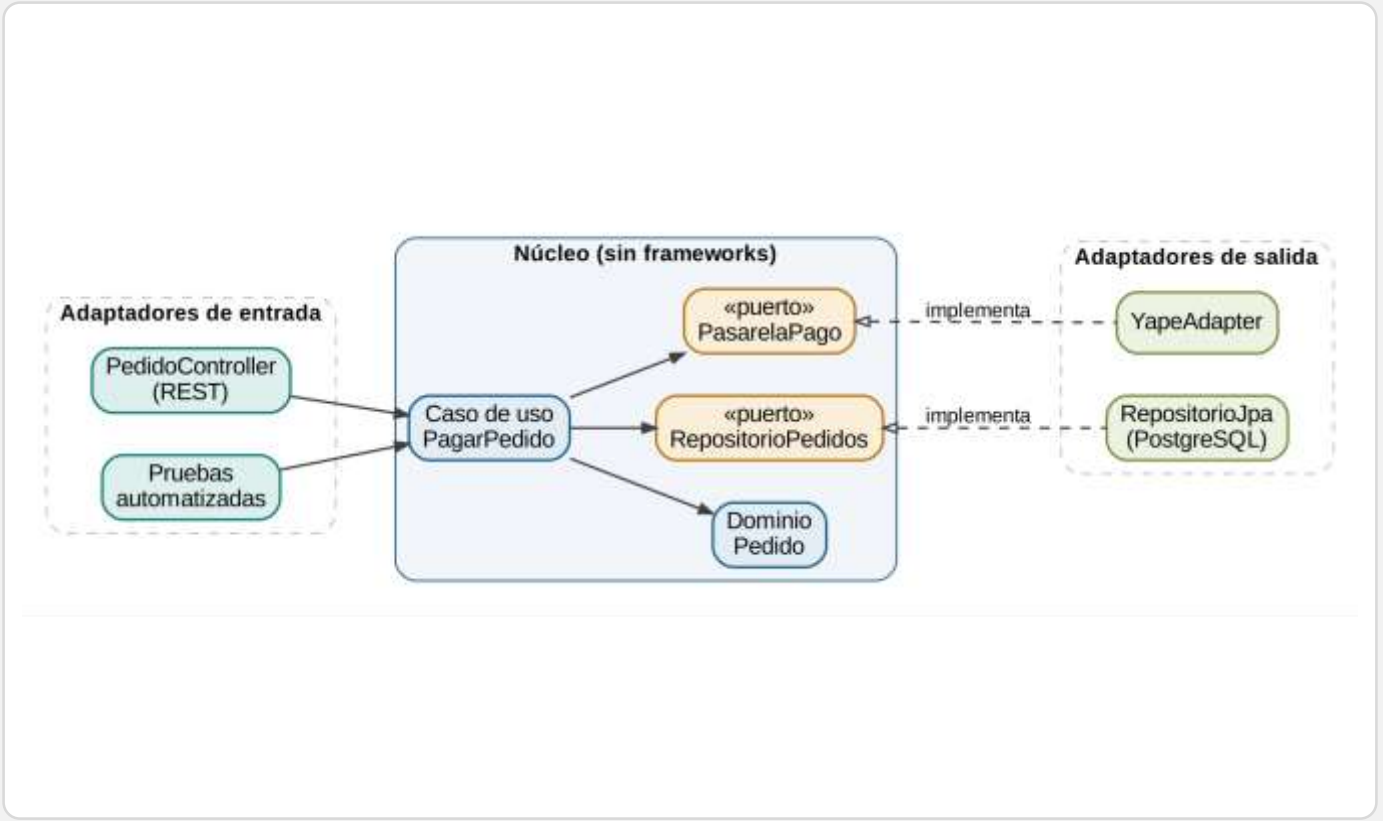


Figura 5. AgroConecta con puertos y adaptadores (Cockburn, 2005).

Hexagonal en Java: el caso de uso

PagarPedido.java

```
package agro.aplicacion;

import agro.dominio.PasarelaPago;
import agro.dominio.Pedido;
import agro.dominio.RepositorioPedidos;

public class PagarPedido {
    private final RepositorioPedidos pedidos;
    private final PasarelaPago pasarela;

    public PagarPedido(RepositorioPedidos pedidos, PasarelaPago pasarela) {
        this.pedidos = pedidos;
        this.pasarela = pasarela;
    }

    public Pedido.Estado ejecutar(long pedidoId) {
        Pedido pedido = pedidos.buscar(pedidoId).orElseThrow();
        boolean aprobado = pasarela.cobrar(pedido.id(), pedido.total());
        pedido.registrarPago(aprobado);
        pedidos.guardar(pedido);
        return pedido.estado();
    }
}
```

Líneas 3–5 · Dependencias

Solo importa puertos y entidades del dominio.

Líneas 11–14 · Inyección

Los adaptadores llegan desde afuera: Spring o una prueba.

Líneas 16–22 · Caso de uso

No sabe si cobra Yape ni dónde se guarda el pedido.

Modelo-Vista-Controlador (MVC)

- **Idea:** separar la interfaz de la lógica.
- **Controlador:** recibe la acción del usuario.
- **Modelo:** datos y reglas de negocio.
- **Vista:** muestra el estado; puede haber varias.

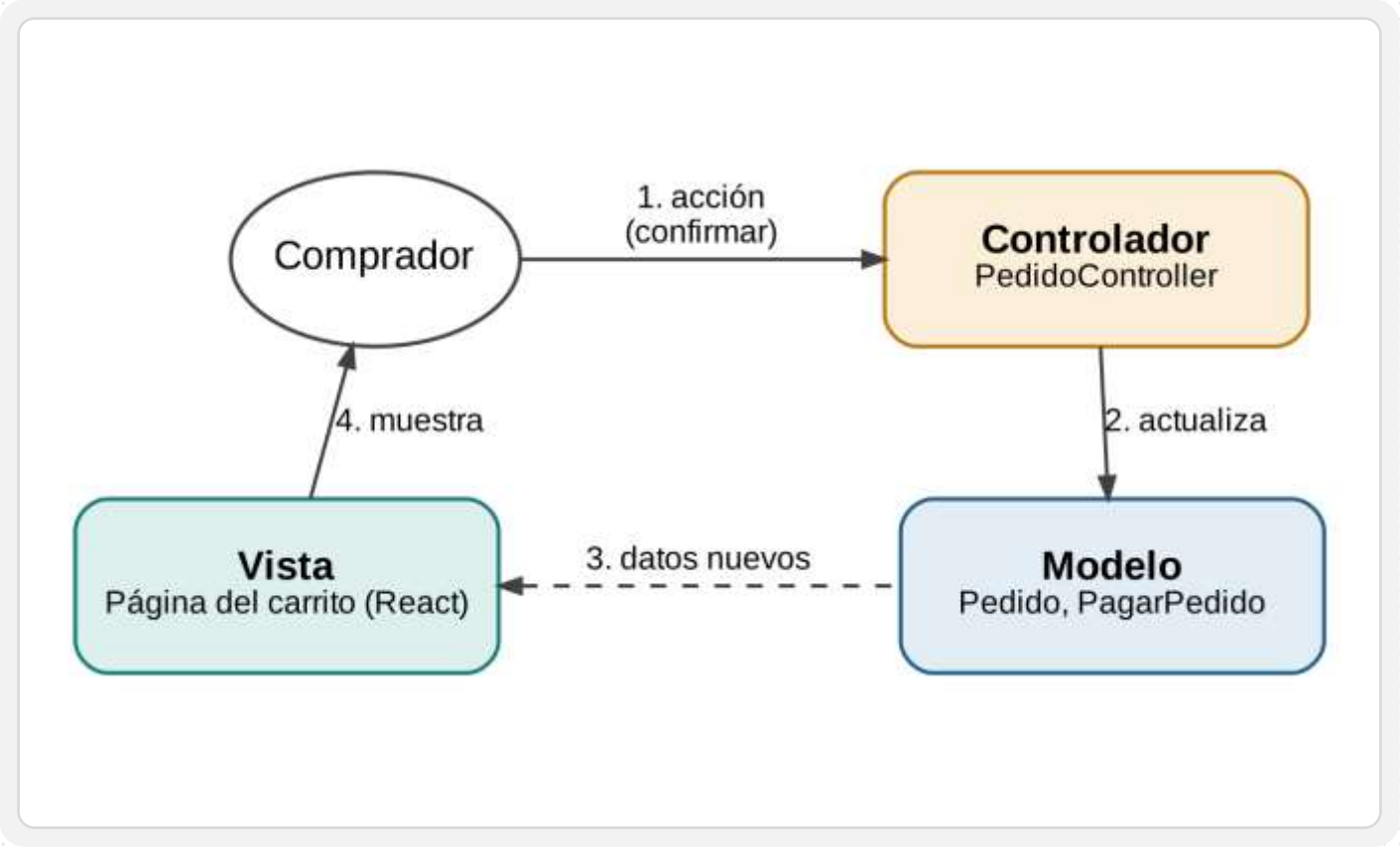


Figura 6. Flujo MVC al confirmar un pedido en AgroConecta.

Publicar/suscribir (eventos)

- **Idea:** un módulo publica un hecho; otros reaccionan.
- **Relación:** es el Observador del Tema 09 a nivel de arquitectura.
- **Favorece:** bajo acoplamiento y extensibilidad.
- **Cuidado:** flujo difícil de seguir; consistencia eventual.

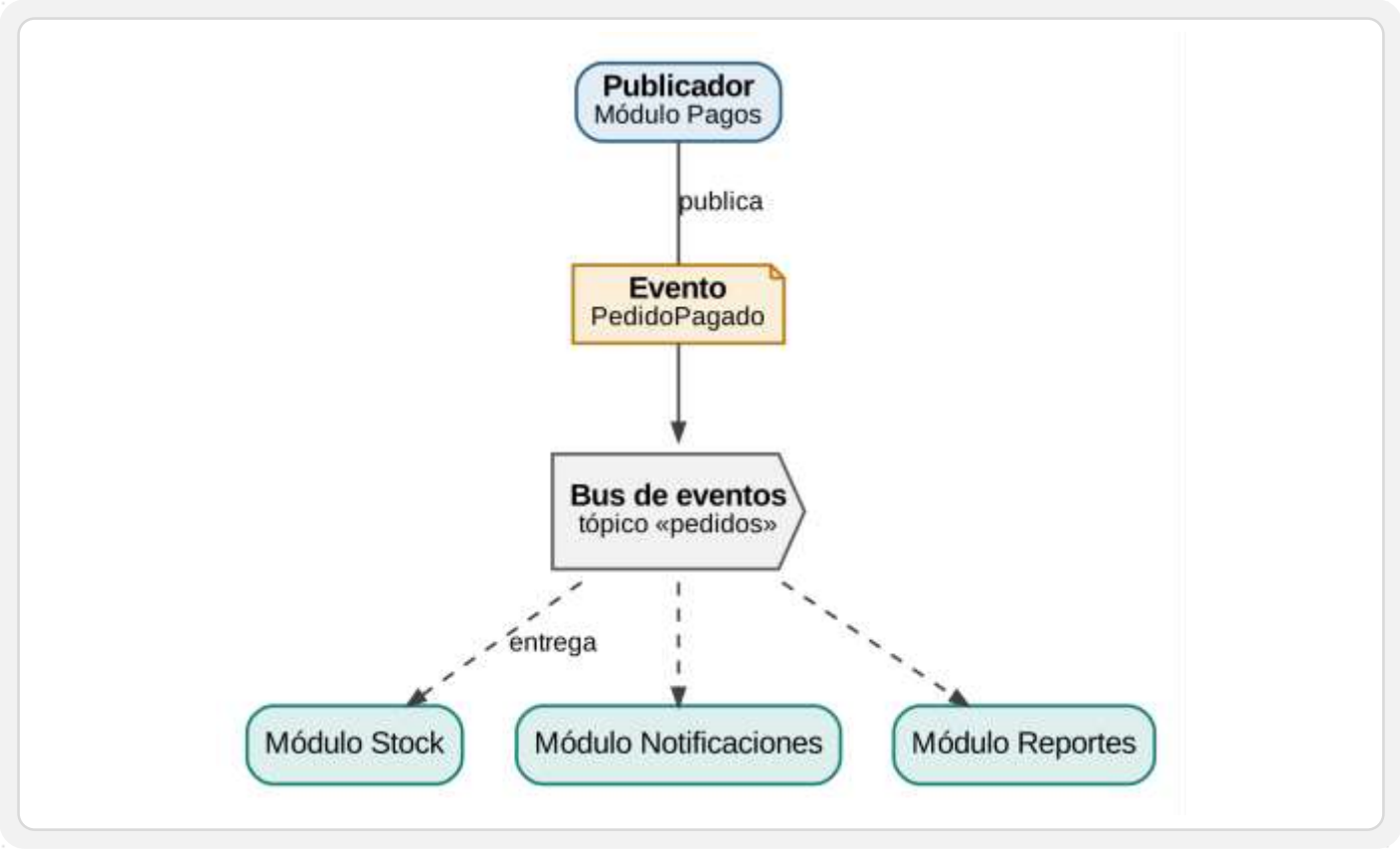


Figura 7. El evento PedidoPagado y sus suscriptores.

Colas de mensajes

- **Idea:** el emisor encola y sigue; un worker procesa.
- **Favorece:** disponibilidad; absorbe picos.
- **Reintentos:** y cola de errores para fallas persistentes.
- **En AgroConecta:** comprobantes de SUNAT y avisos de WhatsApp.

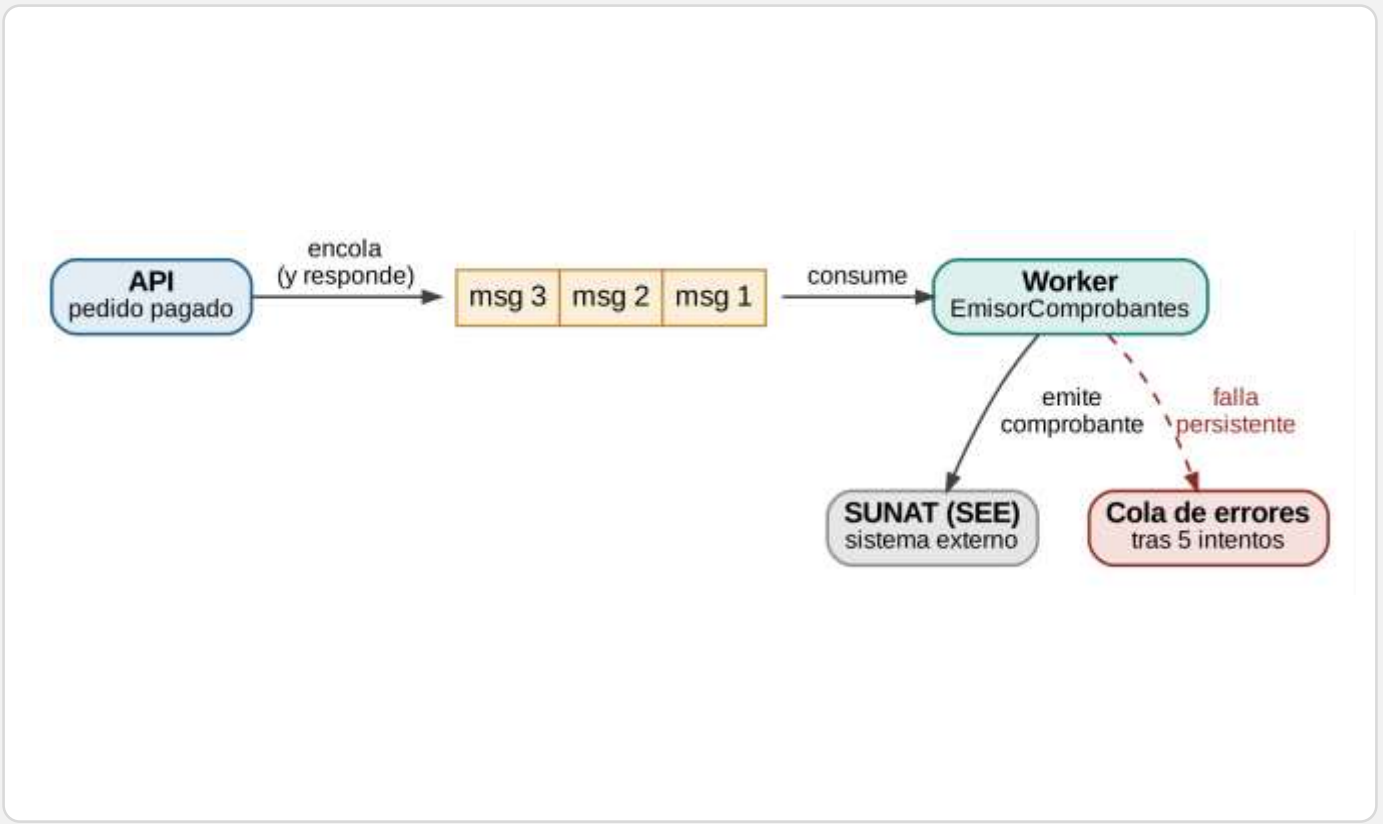


Figura 8. Emisión asíncrona de comprobantes con una cola.

Comunicación síncrona vs. asíncrona

Síncrona (HTTP)

Características
el cliente espera la respuesta del servidor.

Ventajas
simple de programar, depurar y probar.

Limitaciones
si el servidor falla o tarda, el cliente también.

Cuándo usarlo
consultas con respuesta inmediata (catálogo).

vs

Asíncrona (colas y eventos)

Características
el emisor deja un mensaje y continúa.

Ventajas
desacopla en el tiempo, absorbe picos, reintenta.

Limitaciones
consistencia eventual; más infraestructura.

Cuándo usarlo
tareas lentas o externas (SUNAT, WhatsApp).

¿Qué patrón para qué driver?

Patrón	Favorece	Cuesta	Úselo cuando
Capas	Modificabilidad, comprensión	Rendimiento por saltos entre capas	Sistemas de información típicos
Hexagonal	Pruebas, cambio de tecnología	Más interfaces y mapeos	El dominio es valioso y las integraciones cambian
MVC	Separar interfaz y lógica	Controladores que crecen sin control	Aplicaciones web con varias vistas
Publicar/suscribir	Bajo acoplamiento, extensibilidad	Flujo difícil de seguir	Varios módulos reaccionan a un hecho
Colas de mensajes	Disponibilidad, absorber picos	Infraestructura y procesamiento diferido	Tareas lentas o servicios poco confiables
Monolito modular	Simplicidad, un solo despliegue	Escala como un todo	MVP con equipo pequeño (ADR-001)
Microservicios	Escalar y desplegar por separado	Complejidad operativa y de red	Equipos grandes y dominios estables

Los patrones se combinan: AgroConecta es un monolito modular, hexagonal por dentro, con colas para tareas externas.

Antipatrón: la gran bola de lodo

Big ball of mud

Un sistema estructurado al azar, extenso y descuidado, remendado con cinta adhesiva y alambre: una selva de código espagueti.

— Foote y Yoder (1997), "Big Ball of Mud" (traducción propia).

Sin capas claras

Todo depende de todo

Prevenir: reglas automáticas

Tácticas de atributos de calidad

Atributo	Tácticas (Bass et al., 2021)	Aplicación en AgroConecta
Disponibilidad	Detectar: timeout, heartbeat · Recuperar: reintento, redundancia, degradación	Timeout y reintentos al cobrar; pedido PENDIENTE_PAGO
Rendimiento	Controlar la demanda · Gestionar recursos: caché, concurrencia, réplicas	Caché del catálogo (Proxy); 2 réplicas de la API
Modificabilidad	Reducir el acoplamiento: encapsular, intermediarios, restringir dependencias	Puertos y adaptadores; el dominio no importa infraestructura
Seguridad	Resistir: autenticar, autorizar, cifrar · Detectar: auditar	JWT con roles; HTTPS; registro de auditoría
Interoperabilidad	Localizar servicios · Gestionar interfaces: adaptar, orquestar	Adaptador de SUNAT con cola de reintentos
Capacidad de prueba	Abstraer las fuentes de datos · Limitar la complejidad	Adaptadores en memoria para las pruebas

Del escenario a la decisión

- **1. Escenario:** qué debe pasar y cómo se mide.
- **2. Tácticas:** cómo se logra el atributo.
- **3. Patrones:** estructuras que implementan las tácticas.
- **4. ADR:** la decisión, sus alternativas y consecuencias.

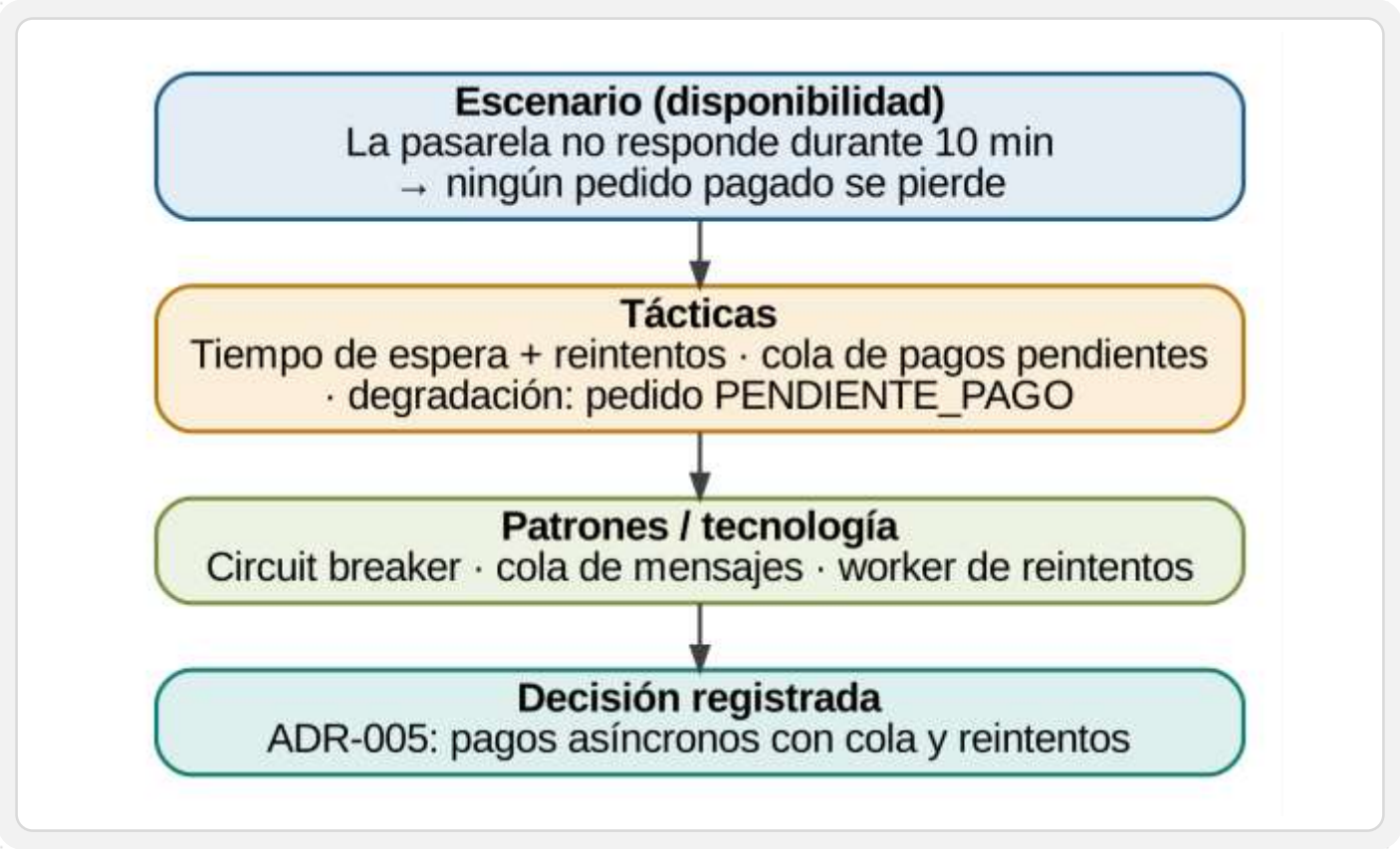


Figura 9. Cadena de decisión para el escenario de disponibilidad de pagos.

Árbol de utilidad

- **Raíz:** la utilidad del sistema.
- **Ramas:** atributos de calidad.
- **Hojas:** escenarios medibles.
- **(I, D):** importancia y dificultad: A, M, B.
- **Primero:** los (A, A); ahí están los riesgos.

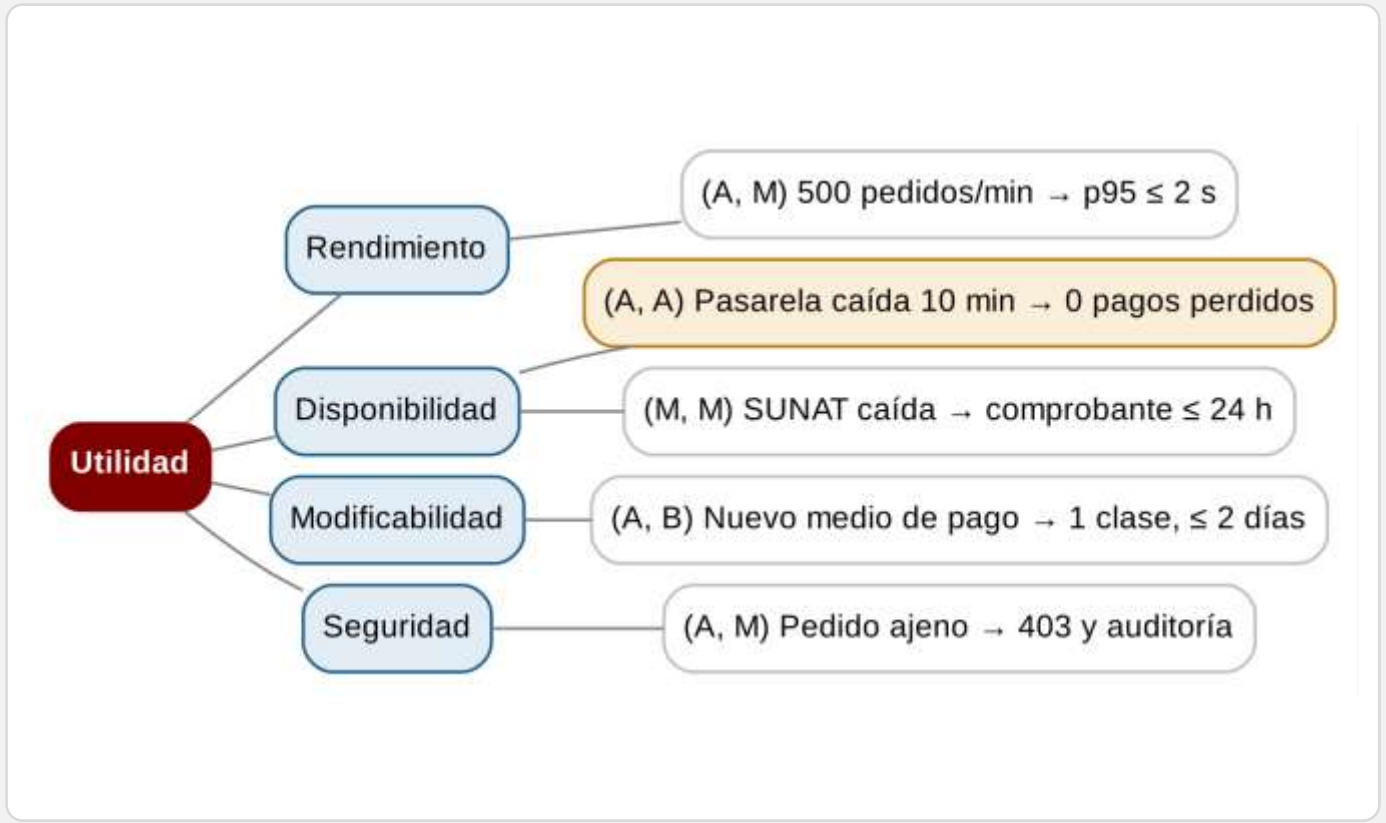


Figura 10. Árbol de utilidad de AgroConecta (ejemplo).

Evaluar con un ATAM ligero



1	2	3	4	5
Objetivos de negocio y escenarios priorizados.	Vistas C4 y ADR vigentes.	Priorizar escenarios por importancia y dificultad.	Recorrer cada escenario (A, A) sobre el diseño.	Registrar riesgos, puntos de sensibilidad y compromisos.

ATAM ligero, adaptado de Kazman et al. (2000) y Bass et al. (2021). En clase: 20 a 30 min por equipo.

Función de aptitud en Java

ReglaDependencias.java (extracto)

```
public class ReglaDependencias {
    static final List<String> PROHIBIDOS =
        List.of("import agro.aplicacion", "import agro.infraestructura");

    public static void main(String[] args) throws IOException {
        List<String> violaciones;
        try (Stream<Path> fuentes = Files.walk(Path.of("src/agro/dominio"))) {
            violaciones = fuentes.filter(p -> p.toString().endsWith(".java"))
                .flatMap(ReglaDependencias::importsProhibidos).toList();
        }
        violaciones.forEach(v -> System.out.println("VIOLACIÓN: " + v));
        if (!violaciones.isEmpty()) System.exit(1);   // el build falla
        System.out.println("OK: el dominio no depende de capas externas");
    }

    static Stream<String> importsProhibidos(Path archivo) {
        try {
            return Files.readAllLines(archivo).stream()
                .filter(l -> PROHIBIDOS.stream().anyMatch(l::startsWith))
                .map(l -> archivo.getFileName() + " -> " + l);
        } catch (IOException e) { throw new UncheckedIOException(e); }
    }
}
```

Líneas 2–3 · La regla

El dominio no puede importar aplicación ni infraestructura.

Líneas 5–14 · Revisión

Recorre las fuentes del dominio y busca imports prohibidos.

Línea 12 · En CI

Si hay violaciones, el build falla. En proyectos reales: ArchUnit.

Una iteración de ADD con IA (y verificarla)

prompt-iteracion-add.txt

```
Actúa como arquitecto de software y aplica ADD 3.0.
Contexto: [drivers, escenarios priorizados, C4 actual].
Objetivo de la iteración: escenario D1 (pasarela caída).

Tarea:
1. Propón 2 alternativas (tácticas + patrones).
2. Compáralas: beneficios, costos y riesgos.
3. Recomienda una y redacta el ADR.
4. Indica qué cambia en el diagrama C4.

Reglas: respeta el ADR-001 (monolito modular);
no agregues servicios que no usamos.
Si falta información, pregunta antes de responder.
```

Líneas 2–3 · Iteración

Un escenario prioritario por iteración, con contexto real.

Líneas 6–7 · Alternativas

Comparar dos opciones evita aceptar la primera.

Después · Verificar

El equipo decide, firma el ADR y actualiza el C4.

04

SECCIÓN

Práctica

25 min · Actividad grupal: una iteración de ADD para RutaColca (documento AG-10).

Actividad grupal: RutaColca en temporada alta

25 min
Duración de la actividad

3–4
Integrantes por grupo

1
Iteración de ADD por grupo

RETO

Su táctica mejora un atributo pero empeora otro. ¿Cuál es el trade-off y cómo lo mitigan?

Descripción del caso

En julio y agosto RutaColca (casos AG-08 y AG-09) recibe diez veces más reservas, la pasarela falla a ratos, las agencias de Chivay pierden conexión y el consorcio quiere aceptar PayPal. Cada grupo diseña una iteración de ADD para el escenario que le toque y la registra en un ADR.

Práctica grupal en clase

ACTIVIDAD GRUPAL

AG-10

Diseño de la arquitectura de RutaColca

Semana 10 · 26–31/10/2026
Duración: 25 min
Modalidad: grupos de 3 a 4

1. **Priorizar · 5 min**
   Árbol de utilidad con los escenarios del documento AG-10.
2. **Diseñar · 10 min**
   Iteración de ADD: tácticas, patrón y cambio en el C4.
3. **Decidir · 5 min**
   ADR con dos alternativas comparadas.
4. **Evaluar · 5 min**
   Otro grupo identifica riesgos y trade-offs.

Rúbrica de la actividad AG-10 (20 puntos)

Criterio	Logrado (4 pts)	En proceso (2 pts)
Escenario	Seis partes y una medida concreta	Medida vaga o partes faltantes
Tácticas y patrón	Adecuados al escenario y justificados	Sin relación clara con el escenario
Alternativas	Dos opciones comparadas con costos y riesgos	Una sola opción
ADR y C4	ADR completo y C4 actualizado de forma coherente	ADR incompleto o C4 sin cambios
Evaluación e IA	Riesgos y trade-offs identificados; IA verificada	Sin evaluación o IA sin verificar

05

SECCIÓN

Cierre

5 min · Ticket de salida, conclusiones y próxima sesión.

Ticket de salida

Responda en tres líneas (2 min)

Escriba un escenario de calidad completo para el pago con Yape: fuente, estímulo, artefacto, entorno, respuesta y medida.

— Respuesta individual en el Aula Virtual.

Escenario | **Medida** | **Táctica**

Conclusiones

01

Escenarios medibles

La arquitectura se diseña para atributos de calidad expresados como escenarios con medida.

02

Patrones y tácticas

ADD elige patrones, tácticas y frameworks en cada iteración y los registra en ADR.

03

Evaluar y verificar

Árbol de utilidad, ATAM ligero y funciones de aptitud. La IA propone; el equipo decide.

Próxima sesión: Tema 11 — Examen Parcial 2 · Guía de laboratorio N.º 08 durante la semana

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley.

Cervantes, H., & Kazman, R. (2016). Designing software architectures: A practical approach. Addison-Wesley.

Cockburn, A. (2005). Hexagonal architecture. https://alistair.cockburn.us/hexagonal-architecture/

Ford, N., Parsons, R., Kua, P., & Sadalage, P. (2022). Building evolutionary architectures (2nd ed.). O'Reilly Media.

Foote, B., & Yoder, J. (1997). Big ball of mud. En Pattern Languages of Programs (PLoP '97).

Kazman, R., Klein, M., & Clements, P. (2000). ATAM: Method for architecture evaluation (CMU/SEI-2000-TR-004). Software Engineering Institute.

Valente, M. T. (2020). Engenharia de software moderna (cap. 7: Arquitetura). Independente. https://engsoftmoderna.info/

Referencias

Cervantes, H., Velasco-Elizondo, P., & Kazman, R. (2013). A principled way to use frameworks in architecture design. IEEE Software, 30(2), 46–53.

Dhar, R., Vaidhyanathan, K., & Varma, V. (2024). Can LLMs generate architectural design decisions? An exploratory empirical study. En IEEE 21st International Conference on Software Architecture (ICSA 2024). https://arxiv.org/abs/2403.01709

Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., & Taibi, D. (2026). Generative AI for software architecture: Applications, challenges, and future directions. Journal of Systems and Software, 231, 112607. https://doi.org/10.1016/j.jss.2025.112607

Jansen, A., & Bosch, J. (2005). Software architecture as a set of architectural design decisions. En 5th Working IEEE/IFIP Conference on Software Architecture (WICSA 2005) (pp. 109–120). IEEE.

Schmid, L., Hey, T., Armbruster, M., et al. (2025). Software architecture meets LLMs: A systematic literature review. arXiv. https://arxiv.org/abs/2505.16697

¿Preguntas?

Gracias por su atención



Ing. Antonio Arroyo-Paz
aarroyop@unsa.edu.pe